Project: The Corporate Blog (TCB)

Developer: Sudhanshu Ghosh

1. Executive Summary

The Corporate Blog is a high-performance, SEO-optimized content management and publishing platform. Engineered with modern web technologies, it provides a secure editorial workflow, rapid static rendering, and robust backend observability, making it an enterprise-grade solution for digital publishing.

2. Architecture & Technology Stack

The application utilizes a decoupled client-server architecture to ensure maximum scalability and developer velocity.

Frontend Setup:

- **Framework:** Next.js (App Router) leveraging server-side rendering (SSR) and Incremental Static Regeneration (ISR) for optimal performance.

- **Language & Styling:** TypeScript paired with Tailwind CSS for rapid, type-safe UI development.

- **State Management:** Zustand is used for lightweight, predictable global state handling.

- **Security & Formatting:** Isomorphic DOMPurify is integrated to sanitize HTML content, preventing XSS attacks.

Backend Setup:

- **Runtime & Framework:** Node.js with Express, written entirely in TypeScript.

- **Authentication:** JWT (JSON Web Tokens) combined with bcrypt for secure password hashing and user session management.

- **Media Management:** Cloudinary integration via Multer handles seamless image uploads and cloud storage.

- **Validation:** Zod schemas ensure strict validation of all incoming payloads before they reach the database layer.

3. Core Features & Workflows

The Publishing Pipeline

The platform implements a strict editorial lifecycle:

1. **Drafting:** Authors create content within a secure dashboard (Status remains `draft`).
2. **Review & Approval:** Editors review the drafts before they go live.
3. **ISR Publishing:** Upon approval, the Next.js frontend triggers Incremental Static Regeneration (ISR). This rebuilds the specific blog post page seamlessly in the background without affecting global site uptime.
4. **Automated SEO:** JSON-LD schemas (Article, FAQ, Breadcrumbs) are injected dynamically into the `<head>` to boost Google Search indexing.

Security & API Resilience

- **Rate Limiting:** Granular rate limits are enforced to prevent abuse. Global API calls are capped at 10 per 15 minutes, publishing at 20 per hour, and searching at 50 per 5 minutes.
- **Sanitization:** The frontend uses `isomorphic-dompurify` to prevent XSS attacks. The backend uses strict payload limits (`10kb`) and HTTP header protection via Helmet.

Observability

- **Sentry Integration:** Comprehensive error tracking and node profiling is active across both the Node backend and Next.js frontend.
- **Custom Metrics API:** An internal `/api/metrics` endpoint monitors total requests, error rates, average response times, and server uptime in real-time.

4. Database Design & Integrity

The database relies on Neon PostgreSQL and strictly enforces data integrity:

- **Soft Deletions:** `deleted_at` timestamps are utilized for both `posts` and `users` to retain data history without permanent removal.
- **Data Constraints:** Database-level checks enforce minimum title lengths (>= 3 chars) and regex-verified email formats.
- **Audit Logging:** A dedicated `publish_audit_logs` table precisely tracks editor activities (e.g., 'PUBLISHED', 'EDITED', 'SOFT_DELETED') alongside the `action_timestamp` and user ID.

5. API Route Structure

The Express backend is cleanly modularized into specific functional routes:

- `/api/auth` - User authentication and JWT management.
- `/api/posts` - Core CRUD operations and draft retrievals.
- `/api/monetization` - Handling ad logic and platform revenue features.
- `/api/search` - Search rate-limited queries for users reading the blog.

Code Repo: https://github.com/sudhanshu9199/The-Corporate-Blog